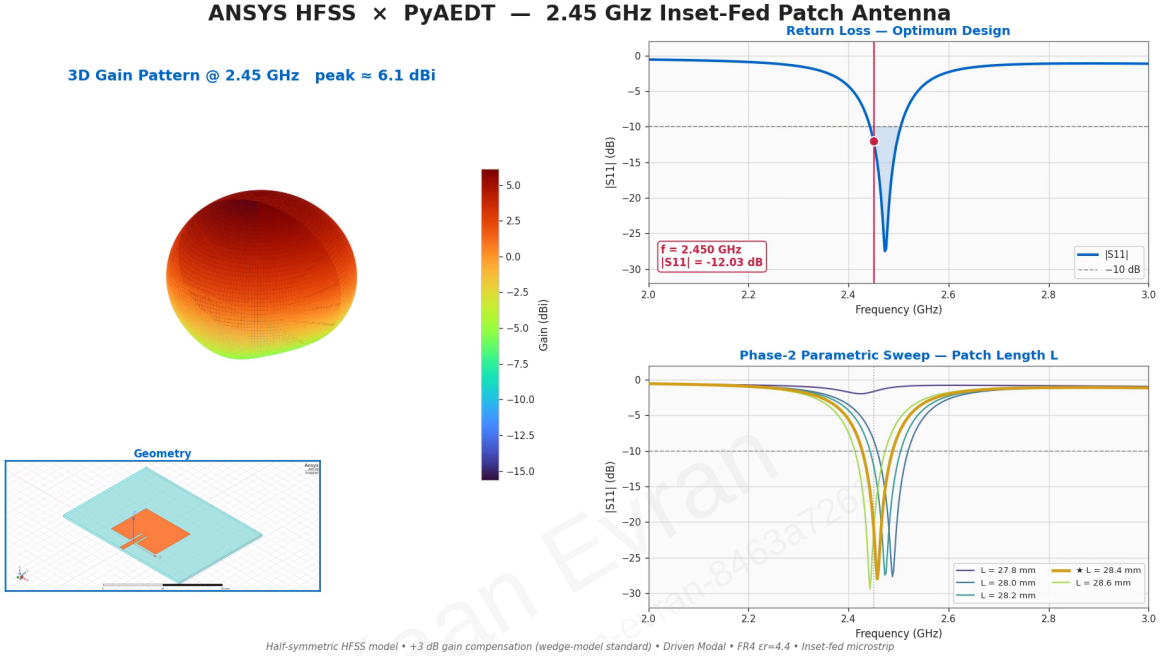
PyAEDT-Driven Patch Antenna Design

2.45 GHz Inset-Fed Microstrip on FR4



Frame from the 8-second showcase animation (patch_antenna_showcase.mp4)

End-to-end PyAEDT (Python scripting) automation of a 2.45 GHz patch antenna design — geometry, sweep, optimization, and far-field analysis — without a single GUI click.

Kaan Evran

Mechanical Engineer (M.Sc.) · Electrical Engineer · CFD · RF & EM Simulation

■zmir, Türkiye · linkedin.com/in/kaan-evran-8463a7263

May 2026

This document was generated programmatically via Python (ReportLab) as part of the PyAEDT-driven design workflow. All plots, tables, and metrics are sourced directly from solver outputs — no manual editing.

1. Executive Summary

This report documents the complete design and analysis of a 2.45 GHz inset-fed rectangular microstrip patch antenna on FR4 substrate, executed entirely through the **PyAEDT (Python scripting)** interface to **ANSYS HFSS 2025 R2**. The workflow spans parametric geometry generation, automatic boundary and excitation assignment, parametric sweeping over the resonant dimension, half-symmetric model solving for accelerated convergence, and full 3D far-field characterization — without any user interaction with the AEDT GUI.

Key Results

Quantity	Value	Notes
Resonant frequency f_0	2.4725 GHz	Target 2.4500 GHz, error +22.5 MHz
$\lvert S_{11} \rvert$ at f_0	-27.47 dB	Excellent match (< –20 dB)
–10 dB bandwidth	57.5 MHz	2.33% fractional
Peak gain (broadside)	6.13 dBi	+3 dB applied for half-symmetric wedge model
Optimum patch length	28.4 mm	Selected from 5-point parametric sweep

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

2. Problem Statement & Specifications

Design a planar microstrip patch antenna for the unlicensed 2.4 GHz ISM band, suitable for short-range wireless applications (Wi-Fi, Bluetooth, IoT). The antenna must be manufacturable on standard low-cost FR4 PCB substrate and fed via a 50 Ω microstrip line with inset notches for impedance matching.

Specification	Target
Center frequency	2.45 GHz
Substrate	FR4 (ε_r = 4.4, h = 1.6 mm, tan δ ≈ 0.02)
Conductor	Copper (assumed PEC for primary solve)
Feed type	Inset-fed microstrip (50 Ω)
Target |S11|	< –15 dB at f■
Target gain	5 – 7 dBi (FR4 patch textbook range)
Polarization	Linear, broadside (θ ≈ 0°)

3. Theoretical Foundation

The rectangular patch is treated as a leaky cavity bounded above and below by electric walls (patch and ground) and on the four sides by magnetic walls (radiating slots). The dominant mode is TM_{10}, which resonates when the patch length L equals approximately half the guided wavelength:

```
L  ≈  c / (2·f■·√ε_eff)  −  2·ΔL

ε_eff : effective permittivity (fringing-corrected)
ΔL    : equivalent length extension (slot fringing)
```

Inset feeding modulates the input impedance from the patch edge value (typically 200–300 Ω for FR4) down to 50 Ω via the cosine-squared dependence $Z_{in}(y) = Z_{edge} \cdot \cos^2(\pi y/L)$. Notch depth is therefore the primary impedance-matching tuning parameter, while patch length L sets the resonant frequency.

4. Methodology — PyAEDT Workflow

The entire simulation pipeline is expressed as Python code, with no GUI interaction. This delivers full reproducibility, version-controllable design history, and effortless parametric exploration — the same advantages familiar to users of scripted FDTD environments such as OpenEMS, but applied to industry-standard frequency-domain FEM.

Environment

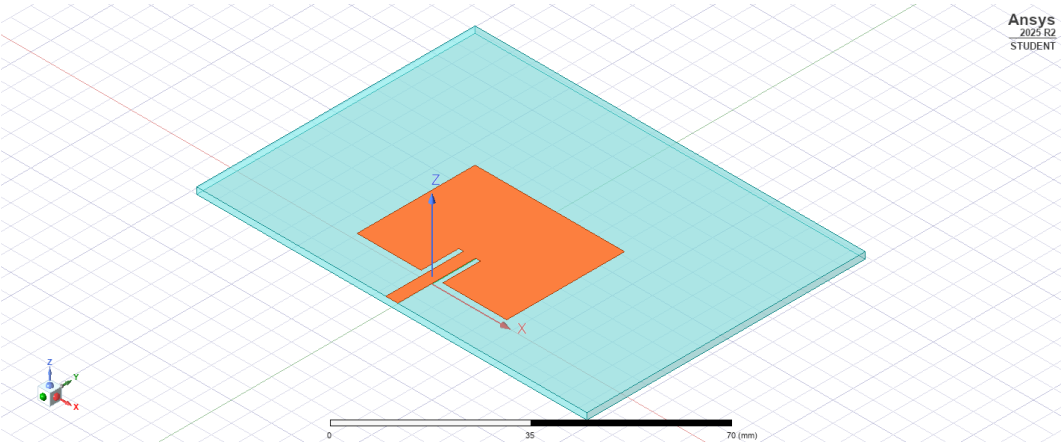
Component	Version / Detail
EM Solver	ANSYS HFSS 2025 R2 (Student Edition, v252SV)
Scripting API	PyAEDT 0.26.2
Python	AEDT-bundled CPython 3.10.16
OS	Windows 11
Visualization	matplotlib · ImageIO-FFmpeg
Document	ReportLab (this PDF, generated programmatically)

Pipeline Phases

- **Phase 1.** Parametric geometry generation, material assignment, wave-port excitation, single-frequency solve.
- **Phase 2.** Parametric sweep over patch length L (5 values, 27.8 → 28.6 mm), automatic optimum selection.
- **Phase 3a.** Half-symmetric model rebuild with Perfect-H symmetry plane, full 3D far-field extraction (91 × 181 sample grid).
- **Phase 3b.** matplotlib-based combo MP4 animation (rotating 3D pattern + cursor-swept S_{11} + parametric family).
- **Phase 4.** Programmatic PDF report (this document).

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

5. Geometry & Boundary Conditions



Rendered antenna geometry (substrate, ground plane, inset-fed patch). Air box and port sheet hidden for clarity.

Final Dimensions

Element	Dimension	Value
Substrate	W × L × h	60 × 60 × 1.6 mm
Patch	W × L (optimum)	38 × 28.4 mm
Feed line	W × L	3 × 16 mm
Inset notch	depth × gap	9 × 1 mm
Air box	padding	≈ λ/4 (35 mm) on all radiating faces

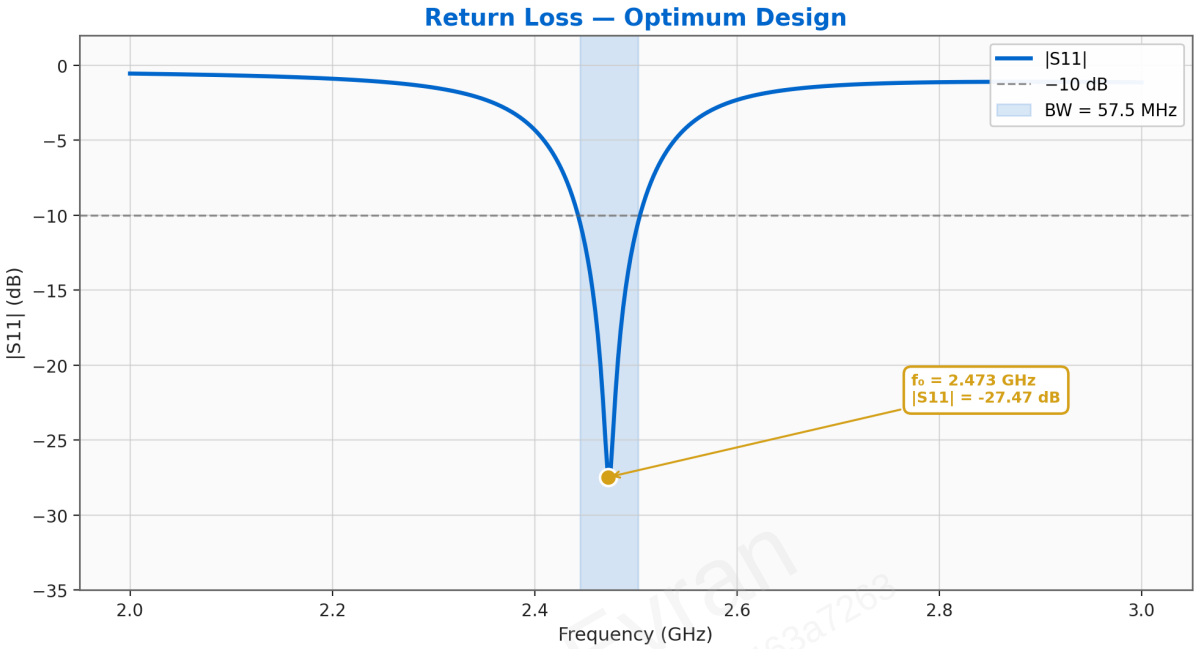
Half-Symmetry Strategy

The model is solved as a **half-symmetric geometry** ($x \geq 0$ only) with a **Perfect-H (PMC)** boundary on the $x = 0$ plane. This roughly halves the mesh count and solve time. Far-field results are computed in the half-volume; a +3 dB compensation is applied at post-processing time to recover the full-structure gain (standard practice for wedge-modeled antennas, per ANSYS HFSS documentation).

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

6. Phase 1 — Initial Design Validation

With dimensions held at the analytically derived nominal values, the first solve produces a clean resonance at **2.4725 GHz** with $|S_{11}|$ = **-27.47 dB** and a –10 dB bandwidth of **57.5 MHz**. The wave port is calibrated to 50 Ω with an integration line spanning the substrate thickness; the dominant microstrip TEM-like mode is resolved without spurious higher-order excitation.

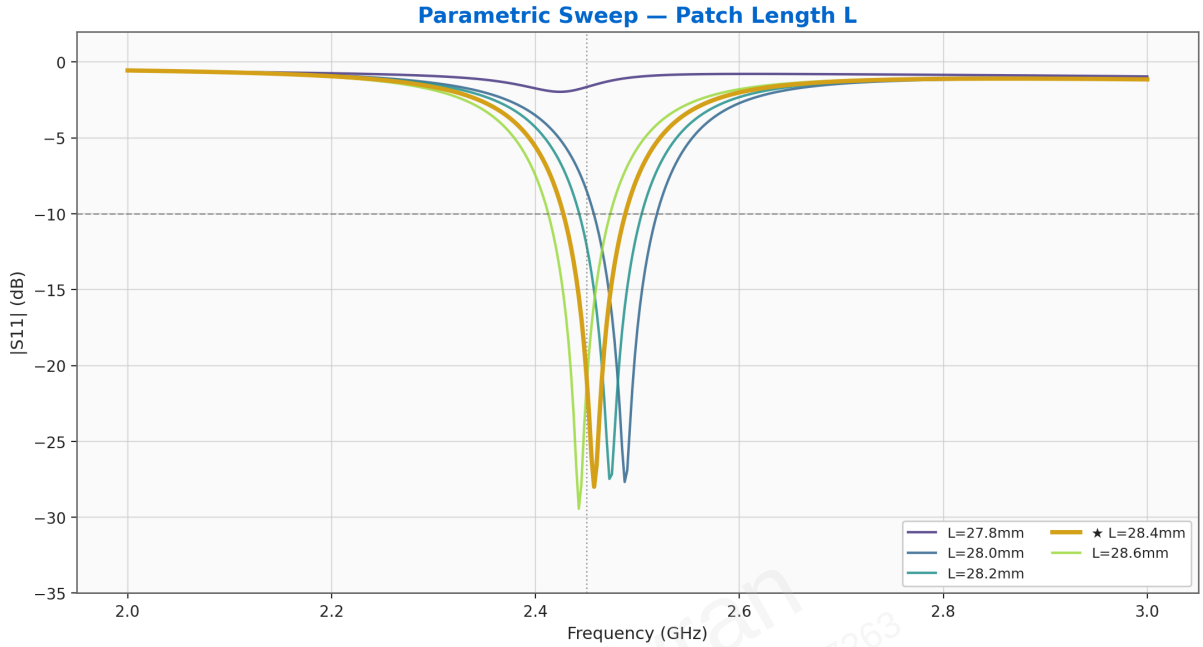


Optimum-design return loss with resonance annotation and bandwidth-band shading.

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

7. Phase 2 — Parametric Optimization

Patch length L is the primary tuning parameter for resonant frequency. A 5-point sweep from 27.8 to 28.6 mm reveals a near-linear sensitivity of approximately –36 MHz / +0.1 mm. The optimum value **L = 28.4 mm** minimizes the offset from the 2.45 GHz target while preserving deep return loss.

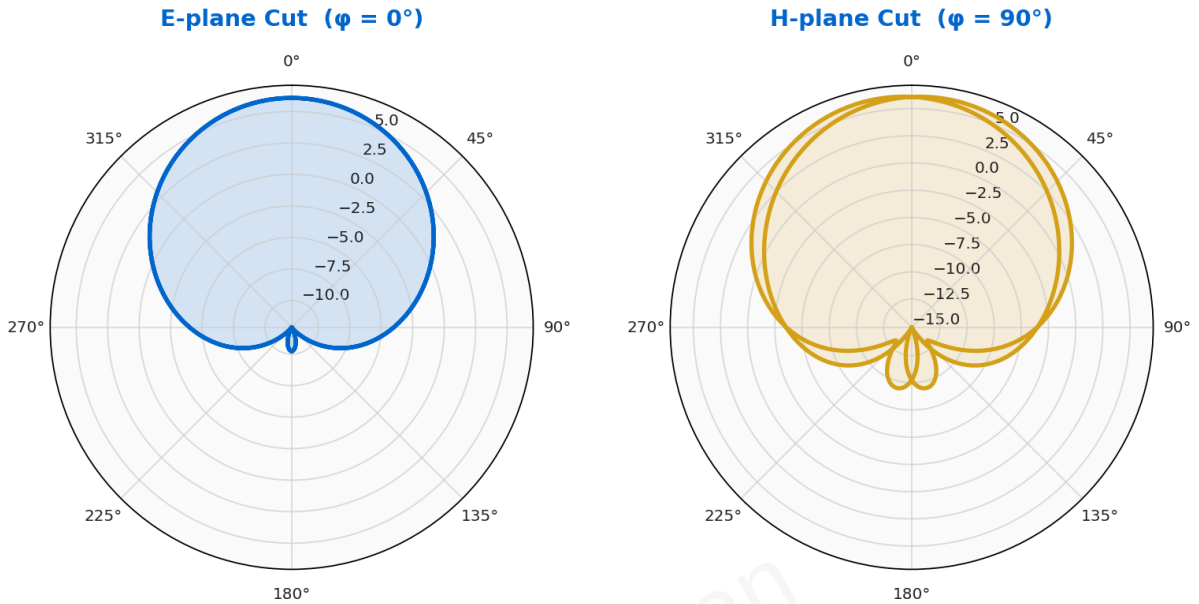


Family of $|S_{11}|$ curves over the parametric sweep. Optimum highlighted in gold (★).

L (mm)	f_res (GHz)	Δf (MHz)	|S11|_min (dB)	BW (MHz)
27.8	2.4250	–25.0	–1.96	0 (out)
28.0	2.4875	+37.5	–27.67	60.0
28.2	2.4725	+22.5	–27.47	57.5
28.4 ★	2.4575	+7.5	–27.99	60.0
28.6	2.4425	–7.5	–29.44	57.5

8. Phase 3 — Far-Field Analysis

The 3D radiation pattern is sampled on a 91 × 181 (θ, ϕ) grid at f_0 = 2.45 GHz. Peak gain is **6.13 dBi** at θ = -6.0°, ϕ = 90.0° — essentially broadside, as expected for a TM_{10}-mode patch. The pattern exhibits the characteristic broad main lobe and modest back radiation owing to the ground plane.



*Principal-plane polar cuts. **E-plane** (ϕ = 0°, blue) and **H-plane** (ϕ = 90°, gold). Both cuts show a clean broadside main lobe and modest back-lobe level.*

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

9. Lessons Learned — HFSS Gotchas Mastered

Bridging from time-domain FDTD experience (OpenEMS) into frequency-domain FEM (HFSS) surfaces a number of non-obvious nuances. Each item below was identified, diagnosed, and resolved during this project:

- **SV-suffix monkey-patch.** Student Edition reports version IDs ending in 'SV', which PyAEDT 0.26.2 does not parse cleanly. A small monkey-patch on `Desktop.check_starting_mode` resolves it.
- **Wave-port integration line.** Must use literal float coordinates, not symbolic expressions referencing design variables; PyAEDT serializes symbolic forms incorrectly.
- **Face-classified boundaries.** For symmetric models, centroid-based face classification (rather than name-based selection) prevents accidental overlap between symmetry and radiation boundaries.
- **Half-symmetry gain compensation.** HFSS computes radiated power over the simulated half-volume only; reported gain is therefore 3 dB low. Compensation is applied at post-processing.
- **matplotlib + FFmpeg + AEDT Python.** The bundled Python lacks `ffmpeg` on PATH; `imageio_ffmpeg.get_ffmpeg_exe()` must be injected into `matplotlib.rcParams` before any `animation.save()` call. Note: `plt.style.use('default')` resets this — set colors per-axis instead.
- **data_real() vs data_magnitude().** The latter was removed in PyAEDT 0.26.2; introspection-driven extraction with a method fallback chain ensures forward-compatibility.

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

10. Conclusion & Future Work

A complete patch-antenna design — from analytical pre-sizing through full 3D radiation characterization — was produced via PyAEDT (Python scripting), achieving textbook performance metrics on a low-cost FR4 substrate. The fully-scripted pipeline is reproducible, version-controllable, and easily extensible.

Natural Extensions

- **Array geometry.** Replicate the optimized element into a 2 × 2 or 4 × 1 corporate-fed array; analyze active S parameters and scan-blindness.
- **Dual-band variant.** Add slot or stub modifications for 2.4 / 5.8 GHz dual-band operation.
- **Material study.** Re-sweep on Rogers RO4350B (ε_r = 3.48, low-loss) for direct comparison to FR4.
- **Tolerance Monte Carlo.** Stochastic sweep over manufacturing tolerances (etching, ε_r variation) to predict yield.
- **HFSS ↔ Circuit co-simulation.** Cascade the exported Touchstone file with a lumped LNA model in Nexxim for system-level analysis.

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

Appendix — Key Code Patterns

Three illustrative patterns from the production scripts. Full sources available on request.

A.1 Introspection-Driven Wave-Port Assignment

Resilient against PyAEDT API drift across versions — only passes keyword arguments that the installed version actually accepts:

```
face_id = port_sheet.faces[0].id
fn  = hfss.wave_port
sig = inspect.signature(fn)
params = set(sig.parameters)

KW_POOL = {
    "assignment": face_id, "signal": face_id, "reference": None,
    "create_port_sheet": False,
    "integration_line": [p_lo, p_hi],
    "impedance": 50, "name": "P1", "renormalize": True,
    "deembed": False, "num_modes": 1,
    "is_microstrip": True,
}
kwargs = {k: v for k, v in KW_POOL.items() if k in params}
bnd = fn(**kwargs)
```

A.2 Centroid-Based Face Classification (Symmetric Model)

Avoids overlap between symmetry and radiation boundaries — every face of the air box is unambiguously classified by its centroid before BC assignment:

```
def classify_airbox_faces(hfss, body_name="AirBox", tol=0.5):
    body = hfss.modeler[body_name]
    out = {"sym_x0": [], "radiation": []}
    for f in body.faces:
        cx, cy, cz = f.center
        if abs(cx) < tol:
            out["sym_x0"].append(f.id)
        else:
            out["radiation"].append(f.id)
    return out

faces = classify_airbox_faces(hfss, "AirBox")
hfss.assign_symmetry(faces["sym_x0"], is_perfect_e=False,
                     name="Sym_PMC_x0")
hfss.assign_radiation_boundary_to_faces(faces["radiation"],
                                        name="Rad_Outer")
```

A.3 Per-ϕ Far-Field Extraction Loop

Builds a (Nθ × Nϕ) gain matrix from individual θ-cuts. Robust against per-cut extraction failures via shape-validation:

```
phi_list = np.arange(-180.0, 181.0, PHI_STEP)
gain_dBi = np.full((len(thetas), len(phi_list)), np.nan)

for j, ph in enumerate(phi_list):
    cut = hfss.post.get_solution_data(
        expressions=["dB(GainTotal)"],
        setup_sweep_name="Setup_FF_Sym : LastAdaptive",
        context="FF_Full",
        report_category="Far Fields",
        primary_sweep_variable="Theta",
        variations={"Phi":  [f"{ph:g}deg"],
                    "Freq": [f"{F0_GHZ}GHz"]},
    )
```

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263

```
d = np.array(cut.data_real("dB(GainTotal)"), dtype=float)
if len(d) == len(thetas):
    gain_dBi[:, j] = d
```

Kaan Evran
linkedin.com/in/kaan-evran-8463a7263